

PYTHON RECORD

NAME: M.sucharitha

SECTION: S3

REGISTER NO.: 11249A233

PYTHON RECORD

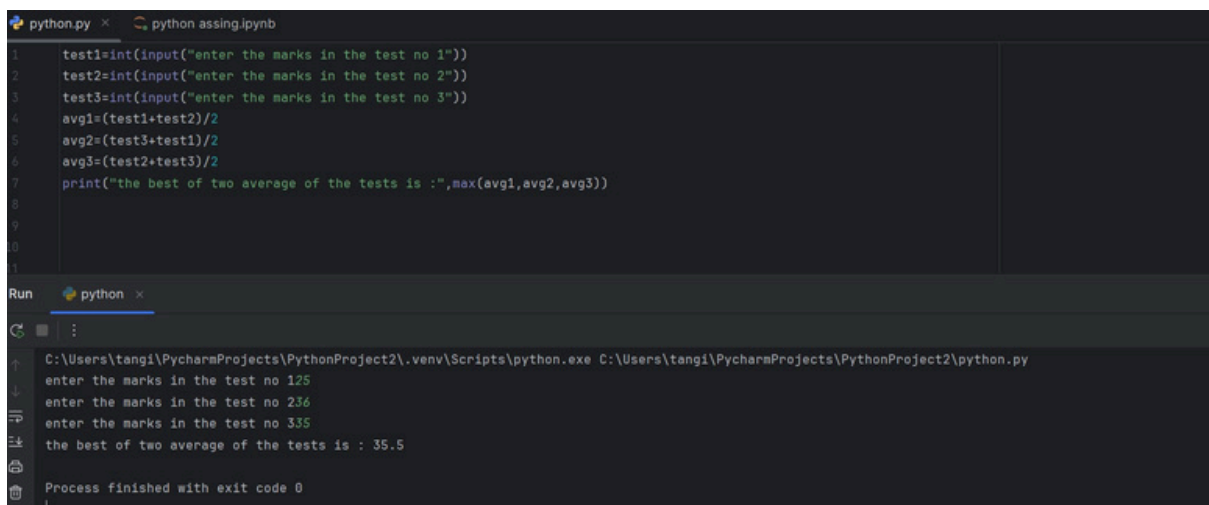
1. AIM:

To calculate the best of two averages out of 3 tests conducted

ALGORITHM:

1. Start
2. Accept 3 test marks from the user
3. Calculate all possible two-test averages
4. Find the highest average
5. Display the best average
6. Stop

PROGRAM AND RESULT:



```
python.py x python assing.ipynb
1 test1=int(input("enter the marks in the test no 1"))
2 test2=int(input("enter the marks in the test no 2"))
3 test3=int(input("enter the marks in the test no 3"))
4 avg1=(test1+test2)/2
5 avg2=(test3+test1)/2
6 avg3=(test2+test3)/2
7 print("the best of two average of the tests is :",max(avg1,avg2,avg3))
8
9
10
11
Run python x
C:\Users\tangi\PchamProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\tangi\PchamProjects\PythonProject2\python.py
enter the marks in the test no 125
enter the marks in the test no 236
enter the marks in the test no 335
the best of two average of the tests is : 35.5
Process finished with exit code 0
```

RESULT:

The best of two averages out of 3 tests conducted was done and output is given.

2. AIM:

To check whether the given is a palindrome or not if yes, check the repetition of the each character

ALGORITHM:

1. Start

PYTHON RECORD

2. Accept a number from the user
3. Convert the number to a string
4. Check if the string equals its reverse → Palindrome or not
5. Initialize a dictionary for digit counts
6. Loop through each digit and count occurrences
7. Display palindrome result and digit counts
8. Stop

PROGRAM AND RESULT:

PYTHON RECORD

```
string=input("enter any thing t check weather it is palindrome or not")
if string==string[::-1]:
    print("yes the given is a palindrome")
    for char in set(string):
        print(f"the character{char} occurred {string.count(char)} times")
else:
    print("no the given is not a palindrome")
```

python x

C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\tangi\PycharmProjects\Pyth

enter any thing t check weather it is palindrome or not1221

yes the given is a palindrome

the character2 occurred 2 times

the character1 occurred 2 times

Process finished with exit code 0

RESULT: The program to check whether the given is a palindrome is done and the output is given

3.

AIM: To print the values in Fibonacci series according to its position using recursive method

ALGORITHM:

1. Start

2. Accept N (>0) from the user

3. If $N \leq 0 \rightarrow$ print error and stop

4. Define function fibonacci(n)

a. Use loop or recursion to calculate series up to N

5. Display the sequence

6. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
def fn(n): 3 usages
    if n <= 2:
        return n - 1
    else:
        return fn(n-1) + fn(n-2)

try:
    num = int(input("Enter a number : "))
    if num > 0:
        print(f' fn({num}) = {fn(num)}')
    else:
        print("Input should be greater than 0")
except ValueError:
    print("Try with numeric value")
```

python x

C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C
Enter a number : 5
fn(5) = 3

RESULT: The program to give the respective Fibonacci series elements was done and the output is given.

PYTHON RECORD

4.

AIM: To write the code for conversions of Binary -> decimal and Octal -> hexadecimal.

ALGORITHM:

1. Start
2. Create a function to convert Binary to decimal
3. Accept a binary number
4. Convert binary → decimal using `int(binary, 2)`
5. Create a function to convert Octal to Hexadecimal
6. Accept an octal number
7. Convert octal → hexadecimal using `hex(int(octal, 8))`
8. Display results
9. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
def binary_to_decimal(binary): 1 usage
    print("conversions over :",int(binary, 2))
def octal_to_hexadecimal(octal): 1 usage
    hexa=int(octal,8)
    print("conversions over :", hex(hexa))
print('enter the choice of conversion \n 1:binary to decimal \n 2:octal to hexadecimal')
choice = int(input())
if choice == 1:
    try:
        value1=input('enter the binary value you want to convert to decimal ')
        binary_to_decimal(value1)
    except ValueError:
        print('Error!!please enter valid binary value')
else:
    try:
        value2=input('enter the octal value you want to convert to hexadecimal ')
        octal_to_hexadecimal(value2)
    except ValueError:
        print('Error!!please enter valid octal value')
```

```
C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\p
enter the choice of conversion
 1:binary to decimal |
 2:octal to hexadecimal
2
enter the octal value you want to convert to hexadecimal 77
conversions over : 0x3f
```

RESULT: The program for converting binary to decimal and octal to hexadecimal is done and the output is given.

PYTHON RECORD

5.

AIM:

Write a Python program that accepts a sentence and find the number of words, digits, uppercase letters and lowercase letters.

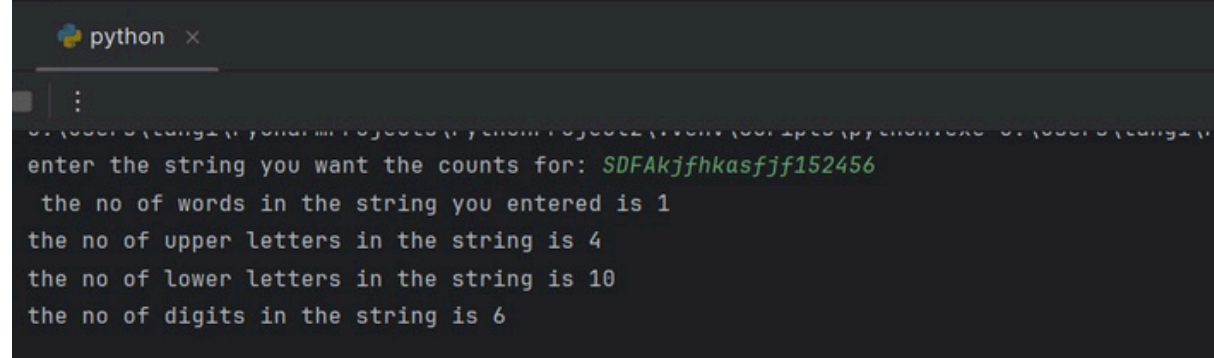
ALGORITHM:

1. Start
2. Accept a sentence
3. Initialize counters for words, digits, uppercase, lowercase
4. For each character in the sentence:
 - a. If space → increment word count
 - b. If digit → increment digit count
 - c. If uppercase → increment uppercase count
 - d. If lowercase → increment lowercase count
5. Display all counts
6. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
string=input("enter the string you want the counts for: ")
words=string.split()
print(f" the no of words in the string you entered is {len(words)}")
upperletters=0
lowerletters=0
digits=0
for ch in string:
    if ch.isupper():
        upperletters+=1
    elif ch.islower():
        lowerletters+=1
    elif ch.isdigit():
        digits+=1
print(f'the no of upper letters in the string is {upperletters}')
print(f'the no of lower letters in the string is {lowerletters}')
print(f'the no of digits in the string is {digits}')
```



```
python x
:
enter the string you want the counts for: SDFAkjfhkasfjf152456
the no of words in the string you entered is 1
the no of upper letters in the string is 4
the no of lower letters in the string is 10
the no of digits in the string is 6
```

RESULT: The program to accept a sentence and find the number of words, digits, uppercase letters and lowercase letters within the sentence is done and output is given.

PYTHON RECORD

6.

AIM: To write a program to compare the similarities of two given strings

ALGORITHM:

1. Start

2. Accept two strings from user

3. Compare character by character

4. Count number of matching positions

5. Calculate similarity percentage = $(\text{matches} / \text{length}) \times 100$

6. Display similarity

7. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
string1=input("enter the first string")
string2=input("enter the second string")
if len(string1)>len(string2):
    short=string2
    print(f"the first string is longer with {len(string1)-len(string2)}more characters")
elif len(string1)==len(string2):
    short=string1
    print(f"the both strings are of the same length with{len(string1)}characters")
else:
    short=string1
    print(f"the second string is longer with {len(string2)-len(string1)}characters")
samechar=[]
for i in range(len(short)):
    if string1[i]==string2[i]:
        samechar.append(string1[i])
print(f"the both strings have {len(samechar)} no of characters at same position")
```

python x

:

C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\tangi\PycharmP
enter the first string*same*
enter the second string*same2*
the second string is longer with 1characters
the both strings have 4no of characters at same position

RESULT: The program to compare the similarities between two given strings is done and output is given.

PYTHON RECORD

7.

AIM: To Write a python program to implement insertion sort and merge sort on lists on random generation

ALGORITHM:

Algorithm (Insertion Sort):

1. Start
2. Accept list of numbers
3. For each element, insert it into the sorted portion of the list
4. Display sorted list
5. Stop

Algorithm (Merge Sort):

1. Start
2. Divide list into two halves
3. Recursively sort each half
4. Merge two sorted halves
5. Display final sorted list
6. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
import random
def insertionSort(array): 1 usage
    for i in range(1,len(array)):
        key = array[i]
        j = i-1
        while j >= 0 and key < array[j]:
            array[j+1] = array[j]
            j-=1
        array[j+1] = key
    return array

def mergesort(array): 3 usages
    if len(array) <= 1:
        return array
    else:
        mid=len(array)//2
        left=mergesort(array[:mid])
        right=mergesort(array[mid:])
        merged=[]
        i=j=0
        while i <len(left) and j<len(right):
            if left[i]<right[j]:
                merged.append(left[i])
                i+=1
            else:
                merged.append(right[j])
                j+=1
        merged.extend(left[i:])
        merged.extend(right[j:])
    return merged

numbers=[random.randint( a: 0, b: 999) for a in range(10)]
```

```
print(numbers)
print(insertionSort(numbers.copy()))
print(mergesort(numbers.copy()))
```

PYTHON RECORD

```
[447, 568, 454, 333, 940, 660, 643, 937, 644, 617]
[333, 447, 454, 568, 617, 643, 644, 660, 937, 940]
[333, 447, 454, 568, 617, 643, 644, 660, 937, 940]
```

RESULT: The program to implement insertion sort and merge sort on the randomly generated series of numbers is done and the output is given.

8.

AIM: To Write a code to recognize a pattern(415-555-4242) of phone numbers without using regular expression and also write the code to recognize the same pattern using regular expression.

ALGORITHM:

Without Regex:

1. Start

2. Accept a string

3. Check pattern: XXX-XXX-XXXX (digits and hyphens in right places)

4. Return True if valid else False

PYTHON RECORD

5. Stop

With Regex:

1. Start

2. Use regex pattern `r'\d{3}-\d{3}-\d{4}'`

3. Match with input string

4. Display result

5. Stop

PYTHON RECORD

PROGRAMANDRESULT(WITHOUT REGEX):

```
def isphonenumner(text): 1 usage
    if len(text)!=12:
        return False
    for i in range(0,3):
        if not text[i].isdigit():
            return False
        else:
            return True
    if text[3]!='-':
        return False
    for i in range(4,7):
        if not text[i].isdigit():
            return False
        else:
            return True
    if text[7]!='-':
        return False
    for i in range(8,12):
        if not text[i].isdigit():
            return False
        else:
            return True
    return True

number=input("enter the telephone number(format- '***-***-***')")
print(isphonenumner(number))
```

```
enter the telephone number(format- '***-***-***')123-456-7890
True
```


PYTHON RECORD

PROGRAM AND RESULT (WITH REGEX):

```
import re

text = input("Enter the phonenumber (format ***-**-****)\n")
pattern = r"\d{3}-\d{3}-\d{4}"

result = re.search(pattern, text)

if result:
    print(result.group(), "is a match ")
```

python x

:

C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users

Enter the phonenumber (format ***-**-****)

123-456-7890

123-456-7890 is a match

RESULT: The program to verify the pattern of telephone using regular expression and without regular expression is done and the output is given.

PYTHON RECORD

9.

AIM: To Develop a python program that could search the text in a file for phone numbers and email addresses

ALGORITHM:

1. Start

2. Open the file in read mode

3. Read the content

4. Use regex to find all phone numbers and email patterns

5. Display all found matches

6. Stop

FILE:

```
the number of himesh is 8432156748 while the number of naresh is 9540232536the number of suresh is 8587542139the number for contacting
```

```
9699875456 and the left over number is to contact some useless fellow i dont know who it is 8563245861
```

```
the email address of gova is tgk@gamil.com the email address for sad@gamil.com the email for contacting some unknown bastard shat@gamil.com
```

PYTHON RECORD

PROGRAMANDRESULT:

```
import re
phone_check = re.compile(r"^\+91\d{10}|\b\d{10}\b")
email_check=re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}')
with open("numbers and emails","r") as f:
    for line in f:
        matches1 = phone_check.findall(line)
        for match1 in matches1:
            print("number in the file",match1)
        matches2 = email_check.findall(line)
        for match2 in matches2:
            print("mail in the file",match2)
```

```
numbers in the file 8432156748
numbers in the file 9699875456
numbers in the file 8563245861
mails in the file tgk@gamil.com
mails in the file sad@gamil.com
mails in the file shat@gamil.com
```

RESULT: The program to get the email addresses and mobile numbers depending on the pattern of the mails and numbers using regular expression is done and the output is given.

PYTHON RECORD

10.

AIM: To Write a python program to accept a file name from the user and perform the following operations:

1. Display the first N lines of the file
2. Find the frequency of occurrence of the word accepted from the user in the file.

ALGORITHM:

1. Start
2. Accept filename from user
3. Open file in read mode
4. Accept N (number of lines) → display first N lines
5. Accept a word from user
6. Count how many times the word appears in the file
7. Display count
8. Stop

PYTHON RECORD

FILE:

the number of himesh is 8432156748 while the number of naresh is 9540232536the number of suresh is 8587542139the number for contacting
9699875456 and the left over number is to contact some useless fellow i dont know who it is 8563245861
the email address of gova is tgk@gmail.com the email address for sad@gmail.com the email for contacting some unknown bastard shat@gmail.com

PROGRAM AND RESULT:

```
filename=input("Enter the filename: ")
try:
    with open(filename,"r") as file:
        lines=file.readlines()
        no_lines=int(input("enter the number of lines you want to see: "))
        for i in range(min(no_lines,len(lines))):
            print(lines[i])
        word=input("enter the word you want to count the repetitions for: ")
        cnt=0
        for line in lines:
            line_s=line.lower().split()
            cnt+=line_s.count(word)
        print(f"the word {word} is repeated {cnt} times in the file")
except FileNotFoundError:
    print(f"File {filename} not found in your system")
```

```
Enter the filename: numbers and emails
enter the number of lines you want to see: 5
the number of himesh is 8432156748 while the number of naresh is 9540232536the number of suresh is 8587542139the number for contacting
9699875456 and the left over number is to contact some useless fellow i dont know who it is 8563245861
the email address of gova is tgk@gmail.com the email address for sad@gmail.com the email for contacting some unknown bastard shat@gmail.com
python sir gave out a lot of works all at once but the only good thing is with the dead lines they all have a lot of difference between them so no need to rush things
enter the word you want to count the repetitions for: email
the word email is repeated 3 times in the file
```

RESULT: The program to accept a file name from the user and perform to perform the following operations:

1. Display the first N lines of the file
2. Find the frequency of occurrence of the word accepted from the user in the file

PYTHON RECORD

11.

AIM: To do a Zip operation on a folder: Develop a program to backing up a given Folder(Folder in a current working directory) into a ZIP File by using relevant modules and suitable methods.

ALGORITHM:

1. Start
2. Import zipfile and os
3. Accept folder name from user
4. Create new ZIP file
5. Walk through folder using os.walk()
6. Add each file to the ZIP archive
7. Close zip file

PYTHON RECORD

PROGRAM AND RESULT:

```
def backup_to_zip(folder):  
    folder = os.path.abspath(folder)  
    number = 1  
    while True:  
        zip_filename = os.path.basename(folder) + '_' + str(number) + '.zip'  
        if not os.path.exists(zip_filename):  
            break  
        number += 1  
    print(f'Creating {zip_filename}...')  
    with zipfile.ZipFile(zip_filename, mode='w') as backup_zip:  
        for foldername, subfolders, filenames in os.walk(folder):  
            print(f'Adding files in {foldername}...')  
            backup_zip.write(foldername)  
            for filename in filenames:  
                file_path = os.path.join(foldername, filename)  
                backup_zip.write(file_path)  
    print(f'\nBackup complete! Folder "{folder}" backed up as "{zip_filename}".')  
    folder_name = input("Enter the folder name you want to back up: ")  
    if os.path.isdir(folder_name):  
        backup_to_zip(folder_name)  
    else:  
        print("Error: Folder not found.")
```

```
C:\Users\tangi\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\tangi\PycharmProjects\Python  
Enter the folder name you want to back up: C:\Users\tangi\Downloads\PYTHON\trails on code  
Creating trails on code_2.zip...  
Adding files in C:\Users\tangi\Downloads\PYTHON\trails on code...  
  
Backup complete! Folder "C:\Users\tangi\Downloads\PYTHON\trails on code" backed up as "trails on code_2.zip"
```

FOLDERS CREATED:

	trails on code_2 Date modified: 19-10-2025 14:56	C:\Users\tangi\PycharmProjects\PythonProject2
	trails on code_1 Date modified: 19-10-2025 14:17	C:\Users\tangi\PycharmProjects\PythonProject2

RESULT: The program to backup a given folder into the zip file is done and output is given.

PYTHON RECORD

12.

AIM: To use the concept of inheritance write a python program to find the area of triangle, circle and rectangle.

ALGORITHM:

1. Start
2. Create base class Shape
3. Create derived classes Triangle, Circle, Rectangle
4. Each subclass implements its own area() method
5. Accept input values and compute area
6. Display results
7. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
import math

class Shape: 3 usages
    def area(self):
        pass
class Triangle(Shape): 1 usage
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self): 1 usage
        return 0.5 * self.base * self.height
class Rectangle(Shape): 1 usage
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self): 1 usage
        return self.length * self.breadth

class Circle(Shape): 1 usage
    def __init__(self, radius):
        self.radius = radius

    def area(self): 1 usage
        return math.pi * (self.radius ** 2)
```

PYTHON RECORD

```
print("Choose shape to calculate area:")
print("1. Triangle")
print("2. Rectangle")
print("3. Circle")

choice = int(input("Enter your choice (1-3): "))

if choice == 1:
    b = float(input("Enter base of triangle: "))
    h = float(input("Enter height of triangle: "))
    triangle = Triangle(b, h)
    print(f"Area of Triangle: {triangle.area():.2f}")

elif choice == 2:
    l = float(input("Enter length of rectangle: "))
    b = float(input("Enter breadth of rectangle: "))
    rectangle = Rectangle(l, b)
    print(f"Area of Rectangle: {rectangle.area():.2f}")

elif choice == 3:
    r = float(input("Enter radius of circle: "))
    circle = Circle(r)
    print(f"Area of Circle: {circle.area():.2f}")

else:
    print("Invalid choice")
```

```
Choose shape to calculate area:
1. Triangle
2. Rectangle
3. Circle
Enter your choice (1-3): 2
Enter length of rectangle: 6
Enter breadth of rectangle: 3
Area of Rectangle: 18.00
```

RESULT: The program to calculate the areas of rectangle, triangle and circle using the concept of inheritance using classes methods in OOPS is done and given.

PYTHON RECORD

13.

AIM: To write a program to store the details of the employees (like name, ID, salary, Dept.) using class and a function to update the salaries of the employees depending on the department

ALGORITHM:

1. Start

2. Create Employee class with name, id, dept, salary

3. Create list of Employee objects

4. Accept department name

5. For each employee in that department, update salary

6. Display updated details

7. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
employees = [  
    {"Name": "Alice", "Employee_ID": 101, "Department": "HR", "Salary": 50000},  
    {"Name": "Bob", "Employee_ID": 102, "Department": "IT", "Salary": 60000},  
    {"Name": "Charlie", "Employee_ID": 103, "Department": "HR", "Salary": 55000},  
    {"Name": "David", "Employee_ID": 104, "Department": "IT", "Salary": 62000}  
]  
  
def display_employees(emp_list): 2 usages  
    for emp in emp_list:  
        print(f"Name: {emp['Name']}")  
        print(f"Employee ID: {emp['Employee_ID']}")  
        print(f"Department: {emp['Department']}")  
        print(f"Salary: {emp['Salary']}\n")  
  
def update_salary_by_department(emp_list, dept, increment): 1 usage  
    for emp in emp_list:  
        if emp["Department"].lower() == dept.lower():  
            emp["Salary"] += increment  
    print(f"Salaries updated for department '{dept}' by {increment}.\n")  
  
print("Employee details before salary update:\n")  
display_employees(employees)  
dept_to_update = input("Enter department to update salary: ")  
increment_amount = float(input("Enter increment amount: "))  
update_salary_by_department(employees, dept_to_update, increment_amount)  
print("\nEmployee details after salary update:\n")  
display_employees(employees)
```

PYTHON RECORD

```
Employee details before salary update:
```

```
Name: Alice
```

```
Employee ID: 101
```

```
Department: HR
```

```
Salary: 50000
```

```
Name: Bob
```

```
Employee ID: 102
```

```
Department: IT
```

```
Salary: 60000
```

```
Name: Charlie
```

```
Employee ID: 103
```

```
Department: HR
```

```
Salary: 55000
```

```
Name: David
```

```
Employee ID: 104
```

```
Department: IT
```

```
Salary: 62000
```

```
Enter department to update salary: it
```

```
Enter increment amount: 5000
```

```
Salaries updated for department 'it' by 5000.0.
```

PYTHON RECORD

```
Employee details after salary update:
```

```
Name: Alice
```

```
Employee ID: 101
```

```
Department: HR
```

```
Salary: 50000
```

```
Name: Bob
```

```
Employee ID: 102
```

```
Department: IT
```

```
Salary: 65000.0
```

```
Name: Charlie
```

```
Employee ID: 103
```

```
Department: HR
```

```
Salary: 55000
```

```
Name: David
```

```
Employee ID: 104
```

```
Department: IT
```

```
Salary: 67000.0
```

RESULT: The program to store the details of the employees using classes and functions and the function to update an employees salary using the department name as a reference is done and the output is given.

PYTHON RECORD

14.

AIM: To Write a python program to find out whether the given input is palindrome or not for both strings and numbers using the concept of polymorphism and inheritance

ALGORITHM:

1. Start

2. Create base class Check Palindrome with method is palindrome()

3. Create subclasses String Palindrome and Number Palindrome overriding method

4. Accept user input (string or number)

5. Call is palindrome() accordingly

6. Display result

7. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
class palindrome:
    def check_palindrome(self,value):
        pass

class string_palindrome(palindrome):
    def check_palindrome(self,value):
        if value == value[::-1]:
            print(f'{value}' is a palindrome (string).")
        else:
            print(f'{value}' is not a palindrome (string).")

class integer_palindrome(palindrome):
    def check_palindrome(self,value):
        value_str = str(value)
        if value_str == value_str[::-1]:
            print(f'{value}' is a palindrome (integer).")
        else:
            print(f'{value}' is not a palindrome (integer).")

print("enter the choice of you want to input \n1.integer\n2.string")
choice = int(input())
if choice == 1:
    user_input = int(input("enter the value of your choice: "))
    obj1=integer_palindrome()
    obj1.check_palindrome(user_input)
elif choice == 2:
    user_input = input("enter the string of your choice \n\t(if it is a sentence enter without using spaces kindly) : ")
    obj2=string_palindrome()
    obj2.check_palindrome(user_input)
```

```
enter the choice of you want to input
1.integer
2.string
1
enter the value of your choice: 1221
'1221' is a palindrome (integer).
```

```
enter the choice of you want to input
1.integer
2.string
2
enter the string of your choice
(if it is a sentence enter without using spaces kindly) : neveroddoreven
'neveroddoreven' is a palindrome (string).
```

RESULT: The program to verify whether the user input is a palindrome or not for both strings and integers using the method of polymorphism and inheritance of OOPS is done and the output is given.

PYTHON RECORD

15.

AIM: To demonstrate a python program to read the data from the spread sheet and write the data into another spread sheet

ALGORITHM:

1. Start

2. Import openpyxl

3. Open spreadsheet using load_workbook()

4. Read data from worksheet

5. Display content

6. Write new data into another cell

7. Save file

PYTHON RECORD

8. Stop

PROGRAM AND RESULT:

```
from openpyxl import Workbook
from openpyxl.styles import Font

wb = Workbook()
sheet = wb.active
sheet.title = "Language"
wb.create_sheet(title="Capital")
lang = ["Kannada", "Telugu", "Tamil"]
state = ["Karnataka", "Telangana", "Tamil Nadu"]
capital = ["Bengaluru", "Hyderabad", "Chennai"]
code = ['KA', 'TS', 'TN']
sheet.cell(row=1, column=1).value = "State"
sheet.cell(row=1, column=2).value = "Language"
sheet.cell(row=1, column=3).value = "Code"
ft = Font(bold=True)
for row in sheet["A1:C1"]:
    for cell in row:
        cell.font = ft

for i in range(2, 5):
    sheet.cell(row=i, column=1).value = state[i - 2]
    sheet.cell(row=i, column=2).value = lang[i - 2]
    sheet.cell(row=i, column=3).value = code[i - 2]

wb.save("demo.xlsx")
sheet = wb["Capital"]
sheet.cell(row=1, column=1).value = "State"
sheet.cell(row=1, column=2).value = "Capital"
sheet.cell(row=1, column=3).value = "Code"
```

PYTHON RECORD

```
ft = Font(bold=True)
for row in sheet["A1:C1"]:
    for cell in row:
        cell.font = ft
for i in range(2, 5):
    sheet.cell(row=i, column=1).value = state[i - 2]
    sheet.cell(row=i, column=2).value = capital[i - 2]
    sheet.cell(row=i, column=3).value = code[i - 2]

wb.save("demo.xlsx")
print("File demo.xlsx created successfully")
srchCode = input("Enter state code for finding capital ")
for i in range(2, 5):
    data = sheet.cell(row=i, column=3).value
    if data == srchCode:
        print("Corresponding capital for code", srchCode, "is", sheet.cell(row=i, column=2).value)
sheet = wb["Language"]
srchCode = input("Enter state code for finding language ")
for i in range(2, 5):
    data = sheet.cell(row=i, column=3).value
    if data == srchCode:
        print("Corresponding language for code", srchCode, "is", sheet.cell(row=i, column=2).value)
wb.close()
```

```
File demo.xlsx created successfully
Enter state code for finding capital TN
Corresponding capital for code TN is Chennai
Enter state code for finding language karnataka
```

RESULT: The program to read data from a given spread sheet and to add the same data into another spread sheet is done and output is given.

PYTHON RECORD

16.

AIM: To write a code to Merge selected pages from Multiple PDFs to a new PDF.

ALGORITHM:

1. Start

2. Import PyPDF2

3. Create PdfMerger object

4. Accept multiple PDF filenames and page numbers

5. Extract required pages

6. Add selected pages to new PDF

7. Save final merged PDF

8. Stop

PYTHON RECORD

PROGRAM AND RESULT:

```
from PyPDF2 import PdfReader, PdfWriter
import os

num_files = int(input("Enter the number of PDF files to merge: "))

pdf_files = []
for i in range(num_files):
    filename = input(f"Enter full path of PDF file {i + 1}: ").strip()
    pdf_files.append(filename)
writer = PdfWriter()

for file in pdf_files:
    if not os.path.isfile(file):
        print(f"'{file}' not found! Skipping...")
        continue

    try:
        reader = PdfReader(file)
        total_pages = len(reader.pages)
        print(f"\n'{file}' has {total_pages} pages.")

        pages_to_add = input(
            "Enter page numbers to include (comma-separated, e.g., 0,2,4): "
        )
```

```
        for page_num in pages_to_add.split(","):
            page_num = page_num.strip()
            if page_num.isdigit():
                page_num = int(page_num)
                if 0 <= page_num < total_pages:
                    writer.add_page(reader.pages[page_num])
                else:
                    print(f"Skipping invalid page number {page_num} in '{file}'")
            else:
                print(f"'{page_num}' is not a valid number. Skipping.")
        except Exception as e:
            print(f"Error reading '{file}': {e}")

output_file = input("\nEnter output PDF file name (with .pdf): ").strip()
with open(output_file, "wb") as out_pdf:
    writer.write(out_pdf)

print(f"\nMerged PDF '{output_file}' created successfully!")
```

PYTHON RECORD

```
Enter the number of PDF files to merge: 2
Enter full path of PDF file 1: de part 1.pdf
Enter full path of PDF file 2: depart 2.pdf
'de part 1.pdf' not found! Skipping...
'depart 2.pdf' not found! Skipping...

Enter output PDF file name (with .pdf): de

Merged PDF 'de' created successfully!
```

FILE CREATED:



RESULT: The program to merge pages from a given number of pdfs is done and output is given.

PYTHON RECORD

17.

AIM: To Write a python program to fetch current weather data from the JSON file

ALGORITHM:

1. Start
2. Import PyPDF2
3. Create PdfMerger object
4. Accept multiple PDF filenames and page numbers
5. Extract required pages
6. Add selected pages to new PDF
7. Save final merged PDF
8. Stop

PYTHON RECORD

FILE:

```
{
  "weather": [
    {
      "id": 800,
      "main": "Clear",
      "description": "clear sky",
      "icon": "01d"
    }
  ],
  "main": {
    "temp": 303.15,
    "feels_like": 305.0,
    "temp_min": 302.0,
    "temp_max": 305.0,
    "pressure": 1013,
    "humidity": 50
  },
  "name": "Bengaluru",
  "wind": {
    "speed": 3.6,
    "deg": 180
  }
}
```

PROGRAM AND RESULT:

PYTHON RECORD

```
import json
filename = input("Enter JSON file name (with .json): ").strip()
try:
    with open(filename, "r") as file:
        data = json.load(file) # Load JSON data as Python dictionary

    city = data.get("name", "Unknown City")
    weather_info = data.get("weather", [{}])[0]
    main_info = data.get("main", {})
    wind_info = data.get("wind", {})
    description = weather_info.get("description", "N/A")
    temp = main_info.get("temp", "N/A")
    feels_like = main_info.get("feels_like", "N/A")
    humidity = main_info.get("humidity", "N/A")
    pressure = main_info.get("pressure", "N/A")
    wind_speed = wind_info.get("speed", "N/A")

    print(f"\nWeather Data for {city}:")
    print(f"Description    : {description}")
    print(f"Temperature     : {temp} K")
    print(f"Feels Like      : {feels_like} K")
    print(f"Humidity        : {humidity}%")
    print(f"Pressure        : {pressure} hPa")
    print(f"Wind Speed      : {wind_speed} m/s")
except FileNotFoundError:
    print(f"File '{filename}' not found.")
except json.JSONDecodeError:
    print(f"File '{filename}' is not a valid JSON file.")

Enter JSON file name (with .json): weather.json
```

Weather Data for Bengaluru:

Description : clear sky

Temperature : 303.15 K

Feels Like : 305.0 K

Humidity : 50%

Pressure : 1013 hPa

Wind Speed : 3.6 m/s

RESULT: The program to fetch data about weather from a json format file is done and the output is given.